



Welcome to a new course where we will delve into the world of animations in the Godot engine. At the end of this course, you will have comprehensive knowledge about animations in Godot. We will start with simple animations and gradually build up to a mini project where we apply the knowledge gained from the basic section.

Course Approach

In this course, we take a specific approach to ensure you understand the concepts in the best possible way. We start with simple animations to learn the basics of the animation player in Godot, creating simple effects and keyframes. Using this foundation, we then progress to a mini project.

Course Structure

The course is divided into two sections: the basics and the mini project.

The Basics

In the basics section, we'll cover:

1. Introduction to the animation player node – the node used to create animations in Godot.
2. Creating an animation that moves and changes the color of a ship.
3. Adding more elements to the animation such as rotation.
4. Changing variables directly from the animation player.
5. Calling functions from the animation player.

The Mini Project

In the mini project section, we aim to create a stylish project using the basics of animation learned in the previous section. The project will contain a character with idle and jump states, a coin with spawn and collect animations, and a visually appealing environment with particles.

Course Requirements

For this course, you only need to have some basic familiarity with Godot and GDScript. Experience in animation is not necessary as we will explain everything from scratch.

About Zenva

Zenva is an online learning academy with over a million students. We offer a wide range of courses for beginners and those looking to learn something new. Our courses are versatile, allowing you to learn in whatever way suits you – from video tutorials and lesson summaries to project files to follow along with the instructor.

That concludes the course introduction. We look forward to seeing you in the first class of this exciting course!

Course Updated to Godot 4.5

We've updated the project files to Godot version 4.5 for this course – the latest stable release.

Please make sure NOT to use newer versions of the software than what we recommend, as the course material provided might not work as expected.

How to Install Version 4.5

You can download the most recent version of Godot by heading to the Godot website (<https://godotengine.org/>) and clicking the “Download Latest” button on the front page. This will automatically take you to the download page for your operating system.



Once on the download page, just click the option for **Godot 4.5**. This will download the Godot engine to your local computer. From there, unzip the file and simply click on the application launcher – no further installation steps are required!

Download Godot 4 for Windows

**Godot Engine****4.5**

x86_64 - 15 September 2025

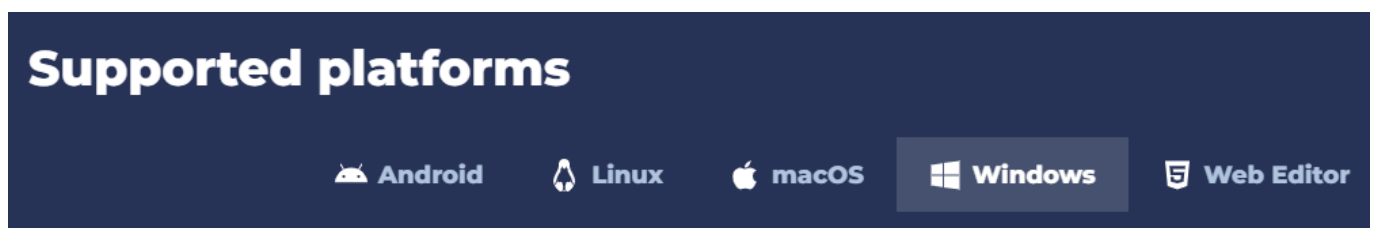
**Godot Engine – .NET****4.5**

x86_64 · C# support · 15 September 2025

Looking for [Godot 3](#), our [experimental releases](#), or a [previous version](#)?



If Godot fails to automatically select the correct operating system for you, you can scroll down to the “Supported platforms” section on the download page to manually select the download version you would like to install:



In this tutorial, we will be setting up a game project in Godot by importing assets and adjusting some basic settings. This guide is intended for beginners and will walk you through the process step-by-step.

Step 1: Download and Extract Assets

Firstly, download the corresponding assets for the game from the course files. These files will be in a zip format. How you extract this zip file may vary depending on your operating system and the specific software you have installed on your device. Regardless, the core process remains the same. Right-click on the zip file and look for options related to WinRAR, 7zip, or any other software that allows you to extract files. Choose the 'extract here' option and wait for the assets to be extracted.

Step 2: Overview of the Assets

Once the extraction is complete, you will find two folders: 'basics' and 'mini project'. The 'basics' folder contains a ship asset that we will use throughout the project. The 'mini project' folder contains various assets such as a background, coins, a player character and tiles for the floor or ground. Each of these assets will be used at different stages of the game development process.

Step 3: Create a new Project in Godot

Next, open Godot and create a new project. Once the project is created, you will need to import the assets into it. To do this, navigate to the project path where you saved the project and copy the assets into it. When you return to Godot, you should see the assets folder with all the extracted files.

Step 4: Organize the Project Files

To keep everything organized, create additional folders within the 'res' folder. You can create separate folders for 'scenes' and 'scripts'. Further, within these folders, create two subfolders named 'basics' and 'mini project'. This will help keep your project files organized and easy to navigate.

Step 5: Adjust the Screen Stretching Settings

Finally, we need to adjust the screen stretching settings to ensure that the game displays correctly at different resolutions. By default, Godot's stretch mode is disabled, which can result in the game breaking when the window size changes. To fix this, go to 'Project' > 'Project Settings' > 'Display' > 'Window'. Under 'stretch mode', select either 'canvas items' or 'viewport'. Either of these options will work for a basic game. With these settings, the game will maintain its aspect ratio across different resolutions, displaying black bars if necessary to keep the aspect ratio intact.

Conclusion

With these steps, you have successfully set up a new game project in Godot, imported the necessary assets, and adjusted the basic settings. You are now ready to start creating your game. In the next tutorial, we will begin building the actual project. Stay tuned!

In this tutorial, we will introduce the Animation Player in Godot and guide you on how to create animations. This feature provides a range of tools to create any kind of animation you want. We will use a sprite image as our subject to animate. We will not be using any other kind of node because the only functionality that we want the sprite to have is to render images and animate.

Setting up the Animation Player

To start, we will create the node as the root node. We will then load our sprite image into our sprite list and name it 'ship'. We will then save this scene in the corresponding scenes folder.

As a child of the 'ship' node, we will add the Animation Player. This is the node used for animating any kind of node. As soon as you add the Animation Player, an animation panel will appear. This panel contains all the tools that the Animation Player provides.

Creating a New Animation

To create a new animation, select the Animation Player and open the animation menu. Here, you can give your animation a name. As soon as you create the animation, you will see a range of options to customize your animation.

Customizing the Animation

First, you will see the animation position in seconds and the length of the animation. You can move this around to adjust the timing of your animation. You can also zoom in or out from the timeline and change the snap.

Next, there are options to play the animation, pause or stop the animation playback, and play the animation backwards from the end. Furthermore, you can change the length of the animation and enable the loop mode. You can also set an animation to autoplay on load.

Animating the Sprite

Finally, to start animating, we use keyframes. For example, you can key the position of the sprite at the start and end of your animation. When you play the animation, there will be an interpolation between the two points, creating a smooth animation.

With the Animation Player in Godot, you can create a variety of animations for your project. Practice with different settings and keyframes to create unique animations.

In this tutorial, we will be creating our first complete animation for a ship in Godot. This animation will involve moving our player from one point to another, and changing its color and opacity as it moves. This is a basic introduction to creating animations in Godot, and we will be starting from scratch by removing any animations we created in previous lessons.

Setting up the Scene

Before we begin creating the animation, we need to set up our scene. Our ship is currently positioned at the top left of the screen, but we want it to be centered. To do this, we need to divide the screen size by two. This can be done by going to the project settings and adjusting the width and height of the viewport. Once we have centered the ship, we can begin creating the animation.

Creating the Animation

To create the animation, we first need to select the animation player and create a new animation. This will open up the animation tab. The animation can be named anything you like, but it's good practice to use a consistent naming pattern. For this tutorial, we are naming our animation "move_change_color".

Keying the Initial Values

After creating the animation, the next step is to define which properties we want to modify. In this case, we are modifying the position and color of the ship. We then need to key the initial values of these properties, which are the values they have in the scene before the animation starts. The initial position is the current position of the ship, and the initial color is its current color.

Key Position

Key Modulate

Modifying the Property Values

Once the initial values have been keyed, we can then modify them at different points in the animation. At the end of the animation, we want to move the ship to the right and change its color to red. We also want to fade it out by decreasing its opacity to zero. These new values are then keyed into the animation.

Key New Position

Key New Modulate

Playing the Animation

After the animation has been created, it can be played by enabling autoplay in the animation player. This will play the animation as soon as the game starts. The animation can also be looped by enabling the loop option. The speed of the animation can be adjusted by changing the speed parameter. A speed of one is the default speed, but it can be increased to make the animation play faster.

Conclusion

In this tutorial, we have created a basic animation in Godot that moves a ship from one point to



another and changes its color and opacity as it moves. We have also learned how to key initial values, modify property values, and play animations. This is a great starting point for creating more complex animations in Godot.

In this tutorial, we will be creating an animation in Godot similar to one from a previous lesson. This animation will involve moving a ship and changing its color simultaneously. Specifically, we will be altering the color from white to red, and gradually reducing its opacity until the ship completely disappears.

Understanding the Basics

For most animations in Godot, the default setting for the ramp is set to continuous. This means there is an interpolation between different keyframes, creating a smooth transition from one point to another. This is known as the default update mode. However, there are instances where you might want to use a different update mode, such as the discrete mode, which we will explore in this tutorial.

Creating a New Animation

First, let's create a new animation. We will call this animation 'move, change color'. After creating the animation, we will follow a similar process to our previous animation. We will move our ship from one point to another and change the color of the ship in the process.

It's important to remember that we need an initial value and an end value for our animation. Initially, our ship will be at a specific position, and by the end of the animation, we will move the ship to the right. Note that the exact values used for the position or color do not have to be the same as the ones used in this tutorial. The result will be similar regardless of the specific values used.

Understanding Discrete and Continuous Update Modes

The update mode we choose impacts how our animation behaves. In a continuous update mode, there is a gradual transition between the initial and end values. This means the color and position of the ship will gradually change throughout the animation.

In a discrete update mode, however, there is no interpolation between the initial and end values. The color and position of the ship will only change at the end of the animation. We can see this difference visually in the animation graph – in continuous mode, the color gradually changes, while in discrete mode, the color only changes at the end of the animation.

Using Discrete and Continuous Modes Together

We can also use discrete and continuous modes together for different properties in the same animation. For example, we can set the position property to discrete and the color property to continuous. This means the color of the ship will gradually change throughout the animation, but the position will only change at the end.

In conclusion, understanding and using the different update modes in Godot can provide a variety of results for your animations. Whether you choose to use the continuous or discrete mode will depend on the specific needs and desired outcome of your animation.

In this lesson, we will delve deeper into the basics of animation in Godot by adding more components to our existing animations. We will discuss properties that can be modified, and how to make our animations look more realistic by adjusting rotations in the animation player.

Creating a New Animation

Let's start by creating a new animation. We will call it 'move, rotate, change color'. The name follows the pattern we have been using throughout our lessons, using verbs to define actions such as 'move', 'rotate', and 'change'.

Our new animation will be similar to our previous ones, but with additional rotation. This will introduce you to more properties that can be modified in animation.

Keying Values

As in our previous animations, we will key some values in position. We will start by keying the initial position, and then key another value at the end of the animation. We will also key the 'modulate' property, which will fade out the animation at the end.

Adding Rotation

Next, we will add rotation to our animation. We will start the rotation at zero degrees, and end it at 90 degrees. We will key these values as well.

You can adjust the timing of the rotation by dragging and dropping the keyframe in the animation timeline. By changing the timing of the rotation, the object in the animation will rotate and then continue moving in the same direction, creating a more realistic animation.

Experimenting with Continuous and Discrete Modes

In this lesson, we also encourage you to experiment with the 'continuous' and 'discrete' modes in the animation player. Continuous mode creates a smooth transition between keyframes, while discrete mode creates a rough transition without any interpolation.

Try changing the track from continuous to discrete and observe the different effects. You can also play around with different colors in the 'modulate' property, and even add more keyframes to create more complex animations.

Challenge

As a challenge, experiment with the following:

- Change the track from continuous to discrete and observe the effects.
- Try out different colors in the 'modulate' property.
- Add more keyframes in the middle of the animation, changing the position, rotation, and modulate as you wish.

By experimenting with these features, you will gain a deeper understanding of animation in Godot and be better prepared for more advanced topics in future lessons.

In this tutorial, we will continue enhancing our animation in Godot by teaching you how to move an object diagonally. So far, we have learned how to change an object's position, color, and rotation. Now, we will add more complexity to our animation by moving the object in multiple directions.

Creating a new animation

Start by creating a new animation. For consistency, name it using the same pattern as your previous animations. In this case, we will name it 'move, rotate, diagonal, change color key'. This name indicates that we will be moving the object, rotating it diagonally, and changing its color.

Setting up the animation

Make the animation longer to give us more time to work with. Reset the speed scale to 1. Begin by resetting all the positions using the reset animation.

Moving the object diagonally

For the position, start with the initial position. At the end of the animation, set the diagonal position. You can easily move the object to the top right by just dragging and dropping the object and keying that value. The animation player will handle the rest, creating a smooth curve between the two values.

Adding rotation

Next, add rotation to the object. The initial rotation will be 1, and the final rotation will be about 90. This will create a more realistic effect, as the object will rotate to the direction it is moving towards. You can also set it to 45 for a more diagonal rotation.

Changing the color

While the object is moving and rotating, we'll also change its color. The initial color will be white, and at the end of the animation, it will change to red with no opacity, making it fade out as it changes to red.

Challenge

Now, here's a challenge for you. Add the other three positions that are missing: top left, bottom left, and bottom right. You will need to change the position value and rotation accordingly for each direction. Experiment with the rotations and positions to get the desired results.

Conclusion

With these lessons, you should now have a good understanding of the animation player in Godot. You've learned how to key any value you want and create basic effects. In the following lessons, we will learn how to modify variables and call functions directly from the animation player, saving you lines of code and making your animations more visual and easier to modify.

In this lesson, we will learn how to modify variables using the animation player in the Godot game engine. This may seem a bit unusual, as typically you would modify variables directly through code. However, the animation player provides an intuitive user interface that allows you to easily modify when a variable is changed and what value it is assigned. It's a great way to visually ensure that everything works as expected. Let's dive in.

Creating an Export Variable

Firstly, if we want to modify a variable through the animation player, we need to declare it as an export variable. Export variables are variables that can be modified in the Godot inspector, as well as in the animation player. If a variable is not declared as an export variable, it can only be modified through scripts.

Let's create a simple example. We will declare a boolean export variable called `isMoving` in a script attached to a ship node:

```
extends Sprite2D

@export var isMoving : bool
```

This variable will act as a flag to indicate whether the player is moving or not.

Modifying the Export Variable in the Animation Player

Now that we have declared our export variable, we can modify it in the animation player. To do this, we add a property track to the animation player and select the `isMoving` variable from the ship script. This will create a new track for the `isMoving` property.

Next, we can insert keyframes to change the value of `isMoving` at specific points during the animation. For example, we could set `isMoving` to `true` at the beginning of the animation and set it to `false` at the end:

At the beginning of the animation:

```
Insert Key -> Time: 0 -> Value: true
```

At the end of the animation:

```
Insert Key -> Time: 1 -> Value: false
```

To confirm that our variable is being modified correctly, we can print its value in the `_process` function of our script:

```
func _process(_delta):
    print(isMoving)
```



When we run the project, we should see the value of `isMoving` change from `true` to `false` as the animation progresses.

Modifying Other Types of Variables

Aside from boolean variables, we can also modify other types of variables in the animation player. For example, we could create an integer export variable called `speed` and modify its value over the course of the animation. This could represent the speed of the ship, for example.

```
@export var speed : int
```

Just like with the `isMoving` variable, we can add a property track for the `speed` variable and insert keyframes to change its value. We could start with a `speed` of `500` at the beginning of the animation and gradually decrease it to `0` by the end:

At the beginning of the animation:

```
Insert Key -> Time: 0 -> Value: 500
```

At the end of the animation:

```
Insert Key -> Time: 1 -> Value: 0
```

By changing the update mode of the track to `continuous`, we can create a smooth interpolation between the start and end values, resulting in a gradual decrease in speed over the course of the animation.

Conclusion

Modifying variables through the animation player can be a useful tool in certain situations, and it can make your code easier to read and understand. It's also a great way to create visually appealing animations and effects in your game. In the next lesson, we will continue exploring the capabilities of the animation player by learning how to call functions.

In this lesson, we will learn how to call functions directly from the animation player in Godot engine. This is a useful feature that can save you some lines of code and time while creating your project. Instead of using a basic animation, we will use a more complex animation that involves moving, rotating, and changing color.

Creating Functions to Call

Firstly, we will create some functions that we will call from the animation player. For this example, we will create a function called 'not_moving' that will print a message when the ship is not moving anymore, i.e., when the animation finishes.

```
func not_moving() -> void:
    print("Not moving")
```

This function doesn't take any parameters and it doesn't return any value. It simply prints the message "Not moving". Remember, this function will work the same way as if you call it by code, so you can modify any variable, call any other function, etc.

Calling Functions from the Animation Player

Now, let's call this function from the animation player. To do this, we need to add a new track in the animation player and select the 'call method track'. We then select the node that has the method that we want to call, in this case, the 'ship'.

After this, we can right click and insert a new key. Here, we will find a variety of methods, including the ones in our script. We can select our 'not_moving' function and call it whenever we want. For this example, we will call it at the end of the animation.

Creating a Function with Arguments

We are not limited to calling functions without arguments. We can also call functions with arguments from the animation player. To demonstrate this, let's create a function that takes an integer as an argument and prints it.

```
func print_number(number : int) -> void:
    print("The number is: " + str(number))
```

Here, 'number' is an integer argument that we convert to a string to print it. We can call this function from the animation player in the same way as before, but this time, we define the argument in the animation player. For example, we can call this function in the middle of the animation and print the number 5.

Calling Inherited Methods

Not only can we call methods that we declare ourselves, but we can also call methods that are directly inherited from the classes. For example, we can call the 'queue_free' method which is inherited from the node class. This method deletes the object, so we can use it to delete the object at the end of the animation.

In summary, the animation player in Godot engine provides a flexible and powerful way to control animations and call functions. We can call any function, whether it's declared by us or inherited from



a class, and we can pass arguments to these functions directly from the animation player. This allows us to create complex and dynamic animations with ease.

In the next lesson, we will start creating a mini project where we will apply the concepts we have learned so far. See you in the next lesson!



Welcome to this new lesson where we will create a mini project, building upon the basics of animations that we have previously learned. In this lesson, we will create a new scene that will serve as the stage for our animations.

Setting up the Scene

We start by creating a new scene where all the animations will take place. For the root node of the scene, we will use a base node. You may wonder why we are using a base node and not a node 2D. The reason is quite simple. Node2D has a transform component that allows us to modify the position, rotation, and scale. However, for our root node, we only need a wrapper or container to hold the different nodes that our project scene will have. We do not need to access position, scale, etc. So, in such cases, it is a good practice to use a base node instead of 2D nodes.

Naming and Saving the Scene

Once you understand the purpose of the base node, you can rename this node to something like 'project'. Save the scene in your scenes folder under the name 'mini project'.

Setting up the Background

Next, we want to add a background to our scene. For this, we add a Sprite 2D node to our scene and load the desired image as its texture. To center the background image, we can use the project settings to get the window width and height, divide them by two, and set these as the position of the Sprite 2D. We can also scale the image to fit the window size.

With these steps, we have set up the basic structure of our scene. In the next lesson, we will create a tile map that will form the floor on which our player will move.



In this lesson, we will be creating a tilemap layer in Godot. This is a crucial step in game development as it allows us to use tiles to create the game environment. Instead of adding and aligning each grass tile separately, tilemaps make this process much easier and efficient.

Creating a Tilemap Layer

To begin with, select the node and add a new tilemap layer. Tilemap layers work with tilesets, which are collections of different tiles that we can use in our game. These tiles are the individual elements that make up our game environment. For our new tilemap layer, we will create a brand new tileset.

Node > Add Child Node > Tilemap

Adding Tiles to the Tileset

After creating the tileset, you can start adding tiles to it. In this example, we have two different tiles (dirt and grass) in two separate files. Drag and drop these files into the tileset panel. Once the tiles are added, you will notice that the grid size does not match the size of our image. To fix this, change the size to match the dimensions of your tiles. In this case, we will use 64 by 64.

Tileset > Add Tile > Select File > Set Size (64 x 64)

Drawing with Tiles

With the tiles added and resized, we can now start drawing our game environment. Select a tile and using the draw tool, you can paint around the viewport. Note that the tile size should match the texture region size. If they don't match, you may encounter issues with the size.

Adding Collisions

By default, tilemaps do not have any kind of collision. This means that they are just images and cannot interact with other objects in the game. To add collisions to our tilemap, select your tilemap layer and add an element in the physics layers. Then, in the tileset, set up the collisions for each tile.

Tilemap > Physics Layers > Add Element > Tileset > Set Collisions

With these steps, you have successfully created a tilemap layer, added tiles to a tileset, drawn your game environment, and added collisions to the tilemap. This is a fundamental process in game development and a great starting point for beginners.



In this tutorial, we will be setting up the player for our game using Godot's RigidBody2D which has built-in physics. This allows us to easily apply forces and impulses to our player, such as jumping and gravity. Let's get started!

Setting up the Player

Our first step is to ensure our player has some physics behavior. This will allow us to apply forces or impulses to our player, such as when they jump. However, while the player is going up, it will also be attracted downwards due to gravity. To simplify this process, Godot provides the RigidBody2D which has built-in physics.

Upon setting up the RigidBody2D, you might notice some warnings indicating that there is no collision shape. This means that our player will not be able to collide with other nodes. To fix this, we need to add a collision shape.

Adding a Collision Shape

Once you've added a collision shape, you might notice another warning indicating that you must assign a shape to your collision shape. This is the shape that will define how your object will collide with others. Before doing this, let's add a sprite to render the image for our player. Once that's done, we can assign the collision shape. For this tutorial, we will be using a rectangle shape and try to match it with the feet of our player to create as accurate a collision as possible.

Centering the Player

Next, we want our player to be centered on the X axis. To do this, go to the project settings, copy the width value, and divide it by two. For the Y axis, you can give it a value like 393. This will position the player in the air, ready to fall down when the game starts.

Adding an Animation Player

The final step in setting up our player is to add an animation player. We won't be creating the animations in this lesson, but it's good to have the animation player set up for future lessons.

Adjusting Physics Settings

Upon running the scene, you might notice that the gravity seems a bit slow. To fix this, increase the gravity value to something like two. This will give a more realistic effect.

Lastly, in the solver settings, turn on Contacts Monitor. This is a property that allows us to emit signals when the player is colliding with other bodies. This will be useful for future lessons when we add other objects for the player to collide with. You can also set the Max Contacts Reported to a value like one, as our player will not be colliding with many objects at the same time.

And that's it! We have successfully set up our player using RigidBody2D and added a collision shape, centered the player, added an animation player, and adjusted the physics settings for a more realistic effect. Stay tuned for future lessons where we will be adding more features to our game!

Welcome to a new lesson where we will be creating player animations in Godot for a simple game that only requires two animations – idle and jump. These animations will coincide with the sprites we have for our player.

Idle Animation

The idle animation will be played when the player is on the ground, essentially colliding with the tilemap. To create this animation:

1. Create a new animation and name it 'Idle'.
2. Select the sprite for the idle frame and key the value.

That's it for the idle animation. The only change we make is to the sprite when the player is idle.

Jump Animation

As the name suggests, the jump animation will be played when the player is jumping or not colliding with the tilemap. Here's how to create it:

1. Create a new animation and name it 'Jump'.
2. Select the sprite for the jump frame and key the value.
3. Add a stretch effect to give the animation some depth. This can be done by scaling the object to 0.5 initially, and gradually increasing it back to 1, its initial scale. This creates a bouncy effect.

```
# Initial scale  
scale = 1
```

```
# Middle of the animation  
scale = 0.5
```

```
# End of the animation  
scale = 1
```

After creating the animations, you might find that they are playing too slow. To fix this, increase the speed scale of the animation player to around 8. This will give you the desired effect.

Although this might look odd at first, it will make more sense once the player's jump is implemented. Stay tuned for the next lesson where we will be doing just that!



Welcome to this new lesson where we will be creating the mechanics of a jump for our player in Godot. We will first need a script in our player, and then we will determine which of the two animations (idle or jump) will play, depending on the current situation. Let's get started!

Attaching a Script to the Player

Firstly, we need to attach a script to our player. Save this script in your scripts folder and ensure the path is correct before clicking 'Create'.

Establishing Jump Conditions

Next, we need to establish when our player can jump. There are two conditions that must be met:

- The player is on the ground: the player should only be able to jump if it is colliding with the tile map (ground).
- The 'jump' key is pressed: a key on the keyboard (in this case, 'Enter' or 'Space') is pressed.

To check for these conditions, we need to declare a variable that will tell us if the player can jump or not. This will be an exported variable, as we will modify its value using the animation player.

Remember, to modify variables in the animation player, they must have the 'export' attribute. This variable will be a boolean, indicating whether the player can or cannot jump.

```
@export var can_jump : bool
```

Checking for Jump Input

In our process delta function, we need to check if the 'jump' key is being pressed and if the player can jump. If both of these conditions are met, we should play the jump animation.

```
func _process(_delta):  
    if Input.is_action_just_pressed("ui_accept") and can_jump:  
        $AnimationPlayer.play("Jump")
```

Note: We use 'just_pressed' to check for the action only once and not all the time that we are keeping pressing the key.

Enabling and Disabling Jumping in Animations

In the idle animation, we should set 'can_jump' to true, as the player should be able to jump when idle. We do this by adding a property track in the animation player and setting 'can_jump' to true at time zero.

In the jump animation, we need to apply an upward force to the player for them to jump. We use the 'apply_central_impulse' method for this, applying a force only on the Y-axis. Remember, in Godot, you move up by decreasing the Y, hence we apply a negative number on the Y-axis. After applying the impulse, we should set 'can_jump' to false, as the player should not be able to jump again while already jumping.

Switching to Idle Animation



We are detecting when the player should jump, but we are not yet detecting when we should be playing the idle animation. We should play the idle animation when the player collides with the tilemap layer. At this point, we should set 'can_jump' to true, allowing the player to jump again.

```
func _on_body_entered(body):  
    if body.is_in_group("TileMapLayer"):  
        $AnimationPlayer.play("Idle")
```

Now, when you play the game and press the 'jump' key (space or enter), the animations should play correctly, and the player should only be able to jump once, even if you keep pressing the key.

And there you have it! With these steps, we have successfully implemented the jumping mechanics for our player in Godot.



In this tutorial, we will learn how to set up a coin in Godot for a mini-game project. The process is very similar to setting up a player with the creation of the setup and different animations. Let's start by creating a new scene.

Creating a New Scene

To create a new scene, we will use an Area2D node. We choose this type of node because we only want to detect collisions with the player, and we don't need any physics behavior. There are several types of nodes that we could use for collisions:

- **RigidBody2D:** This node type comes with integrated physics. However, we don't need any physics in the coin, like gravity or impulse, so this isn't the best choice in this case.
- **CharacterBody2D:** This type is usually used for objects that are moved by script. Since we won't be moving the coin by script, this also isn't the best option.
- **StaticBody2D:** We won't use this type because it doesn't provide a method to detect collisions. StaticBodies are meant to be used for things like walls, roofs, and grounds that won't be moved, hence the name 'Static'.

The most fitting node for our use case is the Area2D node. It's designed for objects that won't be moved by script and won't have physics behavior, but still need to detect collisions.

Adding Sprites and Collision Shapes

Next, we will add sprites to render the image of the coin. We have a variety of coin images that we can randomly choose from later. For now, we'll use the first one. We'll also add a collision shape to the coin. If the coin appears too large, we can scale down both the sprites and the collision shape.

The shape of the coin isn't a perfect rectangle or circle, but we can use a circle for the collision shape and adjust the radius to fit the coin image.

Adding an Animation Player

Finally, we will add an AnimationPlayer to the coin, which we will modify in the next lesson. This completes the setup for our coin.

With these steps, you now have a basic understanding of how to set up a coin in Godot. In the next lesson, we will learn how to add different animations to the coin.



In this tutorial, we will learn how to create animations for an object, specifically a coin, in our game. We will create two animations: a spawn animation and a collision with player animation. The spawn animation will play as soon as the coin is instantiated in the game scene, while the collision animation will play when the coin is collected by the player.

Creating the Spawn Animation

The first step is to add the spawn animation to the animation player. This is done by selecting the animation player and clicking on the 'add' button. Name the animation 'Spawn'. This animation will be played as soon as the coin is instantiated.

For the spawn animation, we will create a simple scale-in effect. This effect will make the coin appear as though it is growing from a smaller size to its original size. The initial size of the coin will be 0.5 and the final size will be 0.8. We will also add a fade-in effect to the animation, starting with no opacity (alpha value of 0) and ending with full opacity (alpha value of 1).

```
// Initial scale and opacity
coin.scale = 0.5;
coin.alpha = 0;

// Final scale and opacity
coin.scale = 0.8;
coin.alpha = 1;
```

If the animation appears too slow, you can increase its speed. In this case, a speed of 4 was found to work well.

Creating the Collision with Player Animation

The next animation to create is the collision with player animation. This animation will play when the coin is collected by the player. For this animation, we will add a rotation effect, a scale-in effect, and a fade-out effect. The rotation effect will cause the coin to rotate 90 degrees, the scale-in effect will cause the coin to grow to a size of 3, and the fade-out effect will cause the coin to gradually disappear.

```
// Initial rotation, scale, and opacity
coin.rotation = 0;
coin.scale = 0.8;
coin.alpha = 1;

// Final rotation, scale, and opacity
coin.rotation = 90;
coin.scale = 3;
coin.alpha = 0;
```

Once the collision animation finishes playing, we want the coin object to be deleted. This can be done by calling the 'queue_free' method of the coin object at the end of the animation.

```
coin.queue_free();
```



And there you have it! You have now created a spawn animation and a collision with player animation for a coin object in your game. These animations add a level of polish and dynamism to the game, improving the overall player experience. Feel free to experiment with different values and effects to create animations that suit your specific game design.

In this lesson, we will learn how to spawn coins in a game and change the sprite when this happens. We will achieve this by creating new scripts and attaching them to the project scene. We will also learn how to instantiate the coin when a function is called. Let's dive right in.

Creating a Reference to the Coin Scene

Firstly, we need to create a reference to our coin scene. To do this, we will create an export variable of type packed scene. This variable will be assigned right away. Here is how you can do this:

```
extends Node
```

```
@export var coin : PackedScene
```

Creating the Spawn New Coin Function

Next, create a function called `spawn\_new\_coin` that will be called when we want to spawn a new coin. The logic of this function is quite simple. We need to get a coin instance and then add it as a child. Finally, we will modify the position of the coin. Here's how:

```
func spawn_new_coin():  
    var coin_instance : Area2D = coin.instantiate()  
    add_child(coin_instance)  
    coin_instance.position = Vector2(576, 248)
```

In this case, we have manually instantiated the coin to check the positions. The X position of the coin is the same as that of the player and the Y position is set to 248.

Calling the Spawn New Coin Function

The `spawn\_new\_coin` function needs to be called at the beginning of the player idle animation. This is because this function is played when the player collides with the tilemap. When the player collides with the tilemap, that is when we want to spawn the coin.

To add the call method track of our project and call the spawn function, you have to firstly save the changes in your script and insert it over there. Move it to time zero. Now, when you play the game, the coin will spawn.

Changing the Coin Sprite

We want to be able to change the coin that is going to be spawned. To do this, we will assign a new script. In this script, we will need an array of textures 2D. We will create a new export variable `coins` that will be an array of textures. Arrays allow us to save multiple values inside of the same variable. We will also create a `sprite\_index` that will determine which index of the coins array we want to use. Here is how you can do this:

```
extends Area2D
```

```
@export var coins : Array[Texture2D]  
var sprite_index : int
```

Then, in the `_ready` function, we will modify the coin texture as soon as this is instantiated. We will assign the `sprite_index` to be equal to `randi() % coins.size()`. This will return a value between 0 and the size of the coins array. The last value is not inclusive. So, we will get either 0, 1, 2 or 3 randomly. This matches the indexes of our array. Now that we have the index assigned, we can modify the sprite texture accordingly:

```
func _ready():  
    sprite_index = randi() % coins.size()  
    $Sprite2D.texture = coins[sprite_index]
```

Now, when you play the game, you will see different coins each time. This is because the sprite index is randomly selected each time the game is played.

And that's it! You have now learned how to spawn coins and change the sprite when this happens in your game. This can be a very useful feature in many types of games, and now you have the tools to implement it yourself.



In this lesson, we will be learning how to add functionality to a game coin such that when the player collides with it, an animation is played. As it currently stands, when the player collides with the coin, a new coin is instantiated on top of the old one, but no other action is taken.

Adding the functionality

To add this functionality, we first need to go to our coin's signals and connect its body entered signal. This will allow us to check if we are colliding with the player. To do this, we will need to create a new group for the player, similar to how we created a group for our 'time up' layer in the player script.

Once you have created the group, you can go back to the coin script and check if the body is in the group layer. If it is, we will play the corresponding animation in the animation player.

```
# In the coin script
if body.is_in_group('player'):
    $AnimationPlayer.play("CollidedWithPlayer")
```

With this code in place, when we collide with the coin, the "CollidedWithPlayer" animation will play.

Testing the functionality

To test this functionality, select the project scene and play the game. You should see that when the player collides with the coin, the coin is deleted, and a new one is instantiated when we collide with the floor. This indicates that everything is working as expected.

And that's it! You have now added functionality to your game coin so that when the player collides with it, an animation is played. This can add a lot of interactivity and fun to your game, making it more enjoyable for players.

In this lesson, we will be adding a particle effect for when a coin is collected in a 2D game using Godot. This will enhance the visual feedback to the player and make the game feel more responsive and engaging. We will create a particle explosion effect that matches the color of the collected coin.

Adding the Particle Node

First, we will add a node for the particle effect. In Godot, we use the `CPUParticles2D` node for creating 2D particles. We'll rename the node to `"coin_collected_particles"`.

Configuring the Particle Node

Initially, the particles are quite bare and need some configuration to achieve the desired effect. Here are the steps to modify the particles:

1. **Disable gravity:** By default, particles have a positive gravity that pulls them downwards. Since we want an explosion effect, we'll set the gravity to zero.
2. **Adjust the scale:** The particles are quite small by default. We can adjust their size by setting a minimum and maximum scale. For example, we can set a minimum of 20 and a maximum of 25.
3. **Set a decreasing scale curve:** We want the particles to start at their maximum size and gradually shrink. This can be achieved by setting a decreasing scale curve.
4. **Add velocity:** To make the particles move, we need to give them an initial velocity. We can set a minimum and maximum velocity to add some randomness to the particle movement.
5. **Adjust the emission direction and spread:** By default, particles are emitted in one direction. We can set the emission direction to zero and increase the spread to maximum, so the particles are emitted in all directions.
6. **Set explosiveness:** The explosiveness property determines how many particles are emitted at once. A value of 1 means all particles are emitted at once, creating a burst effect.
7. **Increase the number of particles:** By increasing the amount of particles, the effect will look more spectacular.
8. **Decrease the lifetime:** By decreasing the lifetime of the particles, the effect will not last as long.
9. **Enable one shot:** Enabling the one shot property will make the particles only emit once when the emitting property is set to true.
10. **Add a color ramp:** To fade out the particles over time, we can add a color ramp to the color property. This will make the particles gradually lose their opacity.

Creating the Emit Function

Next, we will create a function in the particle node's script to control the emission of particles. This function will take a color as an argument, which will be used to set the color of the particles. The function will set the emitting property to true and the color property to the provided color.

```
# CoinCollectedParticles.gd
func emit(coin_color : Color) -> void:
    emitting = true
    color = coin_color
```

Calling the Emit Function

Finally, we need to call the emit function when a coin is collected. We can do this in the coin's script, after the collision animation is played. We will get a reference to the particle node from the parent



node (which is the scene root in this case), and call the emit function with the color of the coin as an argument.

```
# Coin.gd
func _on_body_entered(body):
    if body.is_in_group("Player"):
        $AnimationPlayer.play("CollidedWithPlayer")
        get_parent().get_node("CoinCollectedParticles").emit(colors[sprite_index])
```

With these steps, we have added a particle effect that is triggered when a coin is collected. The color of the particles matches the color of the collected coin, providing visual feedback to the player and enhancing the game experience.



In this tutorial, we will learn how to enhance our game project using landed particles. These particles will be played when we land, during the idle animation. This addition will make our final project more visually appealing. The good news is that we already have some particles created, so we don't have to start from scratch. We can simply duplicate the existing particles and modify them to suit our needs.

Creating the Landed Particles

To start, we will create a copy of the 'coin collected' particles by duplicating them. Let's rename this duplicate to 'landed particles'. We won't need the script from the original particles for this, so we can delete it from the node.

When to Play the Landed Particles

Next, we need to consider when these particles will be played. As mentioned earlier, we want them to be played when we land, which is when the idle animation is played. We can call this function from the idle animation. More specifically, we will be enabling the 'emitting' property.

Positioning the Landed Particles

We also need to adjust the position of the particles. They should be emitted on the ground, not at the point where the original particles were emitted. We can zoom in to position them as accurately as possible.

Modifying the Landed Particles

Now that we have our landed particles, we can start modifying them. We can adjust the gravity, color, and emission shape to make them look more unique and different from the original particles.

- Gravity: The current particles have no gravity and stay in the same place. We can add some force to make them go down.
- Color: The current particles are white, which may not match the color of the player. We can change the color to match the player's color.
- Emission Shape: The current particles are emitted all at the same time from the same point, which gives an explosion effect. We can change the emission shape to a rectangle or line to have the particles in the same line that the player would have over here.

Final Adjustments

After making these modifications, we can make some final adjustments to the number and size of the particles. We can reduce the number of particles and make them smaller to differentiate them from the explosion particles.

Remember, particles are mostly about experimenting and playing around with the values. By tweaking the scale, gravity, color, and other properties, you can fine-tune the final result to get the effect you want. You can also reuse and modify existing particles in your game projects to save time and effort.

Conclusion

This tutorial has shown you how to enhance your game project with landed particles. By reusing and modifying existing particles, you can create a more visually appealing game without starting from scratch. Happy coding!

Congratulations on completing the Godot animations course! This course aimed to provide a comprehensive understanding of the animation player in Godot, starting from the basics and leading up to a mini project. The course was divided into two main parts: the basics and the mini project.

The Basics

The basics section started with a quick introduction to the animation player in Godot. We then proceeded to create simple animations. Here's what we covered:

- Introduction to the animation player: how to create, delete, and duplicate animations.
- Creating simple move and change color animations.
- Adding tracks to the animation for additional movements and rotations.
- Adding diagonal rotation and movement to the animation.
- Changing values inside of variables in the animation player.
- Directly calling functions from the animation player.

These basics were demonstrated using a simple ship animation example.

The Mini Project

In the second part of the course, we used the knowledge gained in the basics section to create a visually appealing mini project. This project featured a character with idle and jump animations and a coin with spawn and collected animations. We also added particles and environment to enhance the visual appeal.

1. Character animations: The idle animation was played when the player wasn't moving, and the jump animation was played when the spacebar or enter key was pressed.
2. Coin animations: The spawn animation involved the coin starting smaller with less opacity and gradually scaling in, while the collected animation involved the coin scaling in, fading out, and rotating 90 degrees.
3. Particles: Particles were added when the player landed and when the player collected the coin.
4. Environment: A background and tiles were added to create the ground with dirt and grass.

This course was brought to you by Zenva, an online learning academy with over a million students. Zenva features a wide range of courses for beginners and those looking to learn something new. The courses are versatile, allowing you to learn in a way that suits you best. You can choose to view video tutorials, read lesson summaries, or follow along with the instructor using included project files.

We hope this course has equipped you with a variety of skills that you can use in terms of animations. We look forward to seeing you in the next course!

In this lesson, you can find the full source code for the project used within the course. Feel free to use this lesson as needed – whether to help you debug your code, use it as a reference later, or expand the project to practice your skills!

You can also download all project files from the **Course Files** tab via the project files or downloadable PDF summary.

Ship.gd

Found in Project/Scripts/Basics

This code defines a `Sprite2D` node with an exported boolean variable `is\_moving` and includes two additional functions: `not\_moving()` which prints “Not moving,” and `print\_number(number)`, which prints a given integer. The `\_process` function is commented out, indicating it might be used for continuous updates in future modifications.

```
extends Sprite2D

@export var is_moving : bool

#func _process(_delta):
#    print(is_moving)

func not_moving() -> void:
    print("Not moving")

func print_number(number : int) -> void:
    print("The number is: " + str(number))
```

Coin.gd

Found in Project/Scripts/MiniProject

This code handles the functionality for a coin object in the game. Upon initialization, it randomly selects a coin texture from an array and sets it for the Sprite2D node. When a player interacts with the coin, it plays a collision animation and emits particles with a color corresponding to the selected coin texture.

```
extends Area2D

@export var coins : Array[Texture2D]
@export var colors : Array[Color]

var sprite_index : int

func _ready():
    sprite_index = randi() % coins.size()
    $Sprite2D.texture = coins[sprite_index]

func _on_body_entered(body):
    if body.is_in_group("Player"):
        $AnimationPlayer.play("CollidedWithPlayer")
        get_parent().get_node("CoinCollectedParticles").emit(colors[sprite_index])
```

CoinCollectedParticles.gd

Found in Project/Scripts/MiniProject

This code extends CPUParticles2D to create a function that emits particles with a specified color. When the `emit` function is called, it starts particle emission and sets the particles' color to `coin\_color`.

```
extends CPUParticles2D

func emit(coin_color : Color) -> void:
    emitting = true
    color = coin_color
```

Player.gd

Found in Project/Scripts/MiniProject

This code handles the jumping mechanics for a 2D character. It checks for jump input and appropriate conditions to trigger a jump animation, and it switches to an idle animation when the character lands on a tilemap layer.

```
extends RigidBody2D

@export var can_jump : bool

func _process(_delta):
    if Input.is_action_just_pressed("ui_accept") and can_jump:
        $AnimationPlayer.play("Jump")

func _on_body_entered(body):
    if body.is_in_group("TileMapLayer"):
        $AnimationPlayer.play("Idle")
```

Project.gd

Found in Project/Scripts/MiniProject

This code spawns a new coin instance from a preloaded coin scene, adds it as a child to the current node, and sets its position to specific coordinates (576, 248). It encapsulates the process of dynamically generating coin objects within the game environment.

```
extends Node

@export var coin : PackedScene

func spawn_new_coin():
    var coin_instance : Area2D = coin.instantiate()
    add_child(coin_instance)
```



```
coin_instance.position = Vector2(576, 248)
```